Copilot Evaluation Harness: Evaluating LLM-Guided Software Programming

Anisha Agarwal
Microsoft
Redmond, USA

Aaron Chan
Microsoft
Redmond, USA

Shubham Chandel
Microsoft
Redmond, USA

Jinu Jang
Microsoft
Redmond, USA

Shaun Miller
Microsoft
Redmond, USA

Roshanak Zilouchian Moghaddam
Microsoft
Redmond, USA

Yevhen Mohylevskyy
Microsoft
Redmond, USA

Neel Sundaresan
Microsoft
Redmond, USA

Michele Tufano
Microsoft
Redmond, USA

arXiv:2402.14261v1 [cs.SE] 22 Feb 2024

Abstract

The integration of Large Language Models (LLMs) into Development Environments (IDEs) has become a focal point in modern software development. LLMs such as OpenAI GPT-3.5/4 and Code Llama offer the potential to significantly augment developer productivity by serving as intelligent, chat-driven programming assistants. However, utilizing LLMs out of the box is unlikely to be optimal for any given scenario. Rather, each system requires the LLM to be honed to its set of heuristics to ensure the best performance. In this paper, we introduce the Copilot evaluation harness: a set of data and tools for evaluating LLM-guided IDE interactions, covering various programming scenarios and languages. We propose our metrics as a more robust and information-dense evaluation than previous state of the art evaluation systems.

We design and compute both static and execution based success metrics for scenarios encompassing a wide range of developer tasks, including code generation from natural language (generate), documentation generation from code (doc), test case generation (test), bug-fixing (fix), and workspace understanding and query resolution (workspace). These success metrics are designed to evaluate the performance of LLMs within a given IDE and its respective parameter space. Our learnings from evaluating three common LLMs using these metrics can inform the development and validation of future scenarios in LLM guided IDEs.

Keywords: Large Language Models, VSCode, Copilot, Code Generation Evaluation

1 Introduction

The continuous evolution of software development practices has led to a growing interest in the integration of cutting-edge technology to enhance developer productivity [10]. One such technology that has garnered considerable attention is the utilization of Large Language Models (LLMs) within Integrated Development Environments (IDEs) [9, 25]. LLMs, exemplified by models like OpenAI's GPT-3.5 [26] and GPT-4 [27], as well as strong open-source models such as Code Llama [35], offer the promise of acting as intelligent programming assistants. In this paper, we introduce the Copilot evaluation harness for comprehensive exploration of the capabilities and potential applications of LLM-guided programming within the context of IDEs, with a particular focus on their adaptability across diverse programming scenarios and languages.

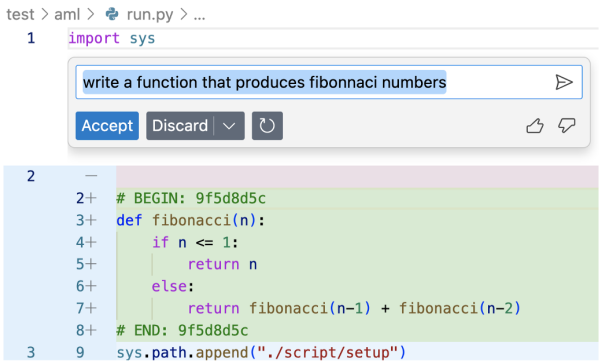

Figure 1. A developer has typed the description of a function, which in this case should generate fibonnaci numbers. The LLM has generated the code for this function highlighted in diff format.

At the core of our investigation lies the evaluation of five major software development scenarios. These scenarios encompass a spectrum of developer tasks, each addressing specific challenges and opportunities:

- **Documentation Generation from Code** (doc): LLMs assist in automating the task of generating documentation from code.

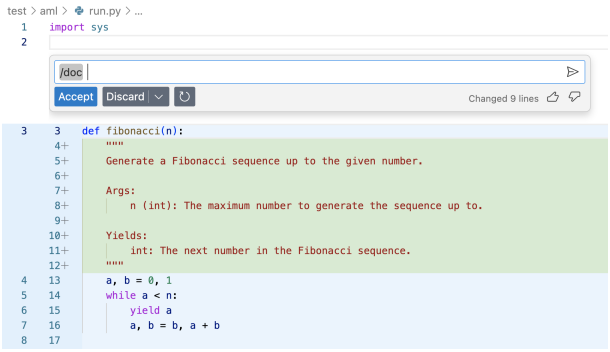

Figure 2. A developer uses /doc to generate documentation for a function that generates Fibonacci numbers. The LLM generates the documentation for this function highlighted in diff format.

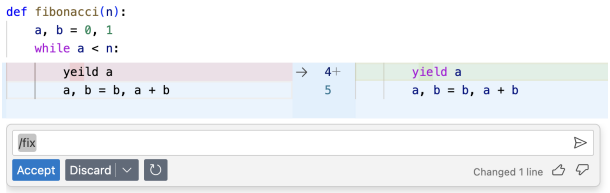

Figure 3. A developer asks the model to fix an error in their fibonacci code, and the model presents the fix (spelling the word "yield" correctly) in diff format.

- **Bug-Fixing** (fix): LLMs play a crucial role in identifying and rectifying warnings and errors raised by static analysis tools.
- **Code Generation from Natural Language** (generate): LLMs generate code snippets from natural language descriptions.
- **Test Case Generation for Code** (test): LLMs are employed to automatically generate test cases for code, aiming to enhance software quality assurance practices.
- **Workspace Understanding and Query Resolution** (workspace): LLMs help developers understand the current project by responding to developer queries according to a comprehension of the codebase in the local workspace.

The metrics in our evaluation harness are designed to evaluate the effectiveness, accuracy, and efficiency of LLM-guided programming interactions across real-world development scenarios. Our framework allows for any IDE to be plugged in and evaluated using our metrics. As such, we provide a system for tuning the IDE parameter space to attain superior LLM-integration outcomes.

While prior work has offered an evaluation harness for code generation [10], a more comprehensive evaluation harness is needed with the new Large Language Models supporting multiple software engineering scenarios in an IDE. There is a wide parameter space to tune and optimize when integrating an LLM with an IDE: how are the prompts to the model phrased [38, 43]? In what order should information be given [23]? How are model responses parsed and inserted back into original code? What context should be provided to the model, in addition to the query [30, 33]? All these factors and more play a role in how well a model can perform within the IDE.

Previous evaluation harnesses leave gaps in this wide space that we seek to cover with the Copilot Evaluation harness. In the HumanEval dataset [10], for example, models are evaluated on their ability to generate functions from docstrings. This is similar to our method generation evalation metric. However, in HumanEval, the test cases are simple, straightforward, algorithmic coding interview style questions. In our test cases, the model must generate functions from real world code, many of which interact with dozens of other methods and files in order to complete a given task. This level of complexity is necessary to properly evaluate the code generation capabilities of state-of-the-art LLMs in a real world setting.

Other works use LLMs themselves to evaluate output [11, 15, 44, 45]. Although this may be effective, there is no guarantee of the model's logic or reasoning, and the natural stochasticity of LLMs makes it difficult to calculate a "true" score for any given test case (i.e. the model may change its answer from run to run). Errors and logic gaps may propagate from the test data into the evaluation results.

With our evaluation framework, we present a new standard of evaluation for model-generated code. Our evaluation harness allows for automatic understanding of how prompt and parameter changes impact performance, across hundreds of test cases spanning a wide range of programming scenarios with real-world code. In this iteration of our work, we discuss detailed results from two of the five metrics outlined above: documentation generation and bug fixing.

We apply our evaluation framework to evaluate the effectiveness of Visual Studio Code, an IDE used by 15 million programmers across the world. Our evaluation spans a spectrum of LLM models, ranging from proprietary models like OpenAI's GPT-3.5 and GPT-4 to openly available alternatives such as Code Llama. We believe that a diverse set of models is essential to provide a holistic perspective on the capabilities and limitations of LLM-guided programming, catering to the needs and preferences of a wide developer audience.

2 Related Work

Below, we explain how our work builds upon and extend the related work on LLMs, Evaluating LLMs, and Evaluating LLMs for software engineering tasks.

2.1 LLMs

Large Language Models (LLMs) [13, 16, 20] are advanced language models with massive parameter sizes that can understand and generate human language. Many of the well-known LLMs such as GPT-3 [14], InstructGPT [28], and GPT-4 [27] leverage the Transformer architecture [40]. In comparison with the traditional machine learning models, LLMs require a large amount of data and very high hardware requirements for training. In return LLMs offer much higher performance than traditional machine learning models when compared their response quality on similar tasks. However, the results from LLMs are less interpretable than the traditional models.

Building on the success of the LLMs researchers have started to explore the advantages of scaling up LLMs. For example, Gropher [32] has 280 billion parameters, Megatron-turing NLG [37] has 530 billion parameters and PaLM [12] has 540 billion parameters outperforming average humans on the BIGbench benchmark [39]. Similarly, researchers also explored fine-tuning LLMs for specific tasks and/or with human feedback [28].

In our study, we examine the performance of three prominent LLMs: OpenAI's GPT-3.5, GPT-4, and CodeLlama on five different software engineering scenarios. We have chosen OpenAI's LLMs as representatives of general-purpose language models applicable to Software Engineering tasks, given their large scale and proprietary nature. In contrast, we have included CodeLlama as an illustration of an open-source, smaller, and optimized model fine-tuned specifically for code-related applications.

2.2 Evaluating LLMs

Previous work has evaluated the effectiveness of LLMs from various angles including performance in natural language tasks, reasoning, robustness, safety, etc [8]. For example, when it comes to sentiment analysis, [22] and [31] showed that LLMs perform much better than traditional sentiment analysis models. Similarly, [21] evaluated ChatGPT's performance on a range of tasks including answering questions, text summarization, code generation, reasoning, and addressing ethical issues.

Unlike traditional machine learning models where k-fold cross validation was a common evaluation process, LLMs are often evaluated using static data sets. Common dataset for evaluating LLMs include: GLUE [42], SuperGLUE [41], BIGBench [39], Massive Multitask Language Understanding (MMLU) [18], Ethics Benchmark [17], and others.

In this paper, we depart from conventional language-based metrics, such as BLEU, commonly employed in previous studies. Instead, we devise metrics tailored specifically for Software Engineering and the tasks under consideration.

2.3 Evaluating LLMs for Software Engineering Tasks

LLMs have been widely used in various software engineering tasks, such as code generation, code summarization, code completion, code search, code documentation, code review, bug detection, and software testing. However, evaluating the effectiveness and efficiency of LLMs for SE tasks is not a trivial problem, as there are many factors and challenges involved. In this subsection, we review some of the existing works that have proposed or applied different evaluation methods and metrics for LLMs for SE tasks.

One of the most comprehensive works is the paper [19], which provides a systematic literature review on the intersection of LLMs and SE, covering various aspects such as data collection, preprocessing, application, optimization, evaluation, and prompt engineering. The paper also categorizes and compares different LLMs that have been used in SE tasks, such as GPT-3, CodeBERT, and GraphCodeBERT, and analyzes their strengths and weaknesses. The paper also identifies the current challenges and future directions for LLMs for SE.

CodeXGLUE [24], is a comprehensive evaluation platform for LLMs in Software Engineering tasks. CodeXGLUE includes a benchmark dataset with 14 tasks covering code intelligence scenarios and provides baseline models like CodeBERT and CodeGPT. It aims to stimulate research and development in LLMs for SE, offering a diverse dataset for various programming languages and tasks. CodeXGLUE's evaluation metrics, both automatic and human-based, along with a leaderboard and online platform, facilitate fair comparisons between models.

One of the first works that evaluated LLMs for code considering code execution and test cases is the paper [10], which introduces HumanEval, a benchmark dataset and a challenge for measuring the functional correctness of LLMs trained on code. HumanEval consists of 164 hand-written programming problems in Python, each with a function signature, a docstring, a body, and several unit tests. The problems cover various topics, such as language comprehension, algorithms, and simple mathematics, and some of them are comparable to simple software interview questions. The goal of HumanEval is to measure the ability of LLMs to synthesize programs from docstrings that pass the given test cases.

In our research, we build upon the foundation laid by prior works in the literature, seeking to enhance their contributions. Like HumanEval, we incorporate considerations of code execution and test cases, but we extend both in terms of the breadth of SE tasks addressed and the refinement of evaluation metrics. In contrast to HumanEval, our evaluation encompasses large and real-world codebases. Furthermore,

our emphasis is on developing a comprehensive evaluation framework for LLM-guided programming within IDE interactions, with a particular focus on their practicality across diverse programming languages and scenarios.

3 Evaluating LLM-Guided Software Programming

In addition to HumanEval [10], match-based metrics such as BLEU [29] or Code-BLEU [34] are commonly adopted to benchmark LLM's performance in software engineering tasks. As LLMs become more ubiquitous and powerful, many researches use LLM models themselves to evaluate LLM output [11, 15, 44, 45]. However, previous work suggests alternative metrics such as functional correctness better reflect success of generative models in code generation [10], code translation[36], and other tasks. Building upon previous work in this area, we expand the HumanEval harness and evaluate IDE-integrated model competency in the five software engineering tasks listed above.

3.1 Documentation Generation from Code (doc)

This task involves generating documentation for a method. Figure 2 shows an example in VS Code IDE. In this case the developer asks the LLM to generate a documentation for a Fibonacci function using /doc.

3.1.1 Metrics. In this scenario, we consider a docstring generation to be successful if the location, format, and coverage of the generated text is correct. We report the following metrics for this scenario:

- *Syntax Correctness*: We check that the docstring has been inserted into the code in such a way that it does not disrupt the syntax of the file with its addition.
- *Format Correctness*: If the documentation comment is placed in a syntactically acceptable manner for the given language, we further check for the correctness of documenting the return statement, function arguments with their types, function name, and whether a function description was written.

3.1.2 Evaluation Procedure. We begin with a set of methods. For each method, we provide the method's signature and body to the LLM as context. We then prompt the LLM with a request to generate documentation for the method, and return the input function with the generated docstring inserted in the correct location within the function .

After the LLM generates the documentation and the generated docstring is inserted into the code file, we evaluate the syntax correctness of the file with the generated docstring, as well as the correctness of the docstring itself.

3.2 Bug-Fixing (fix)

This task involves using LLMs to fix bugs identified by static analysis tools, with an expectation that the resulting fixed code will have fewer errors overall than the original code. We use the following static analyzers:

- javascript: eslint [2];
- ts: eslint [2], tsc (typescript compiler);
- python: pylint [4], pyright [5];
- java: spotbugs [7];
- c#: roslyn [6];
- cpp: clang [1].

If the original error is fixed but another error is introduced in its place, the test case will fail.

Figure 4 shows an example in the VS Code IDE. A programmer has an error because of a misspelling of the word "yield", and the model corrects the error.

3.2.1 Metrics. In this scenario, we consider a bug fix to be successful if the resulting code is syntactically correct and the corresponding static analysis warning or error has disappeared.

- *Syntax Correctness*: we confirm that the code file with the bug fix remains syntactically correct.
- *Fix Rate*: we check that an existing static analysis warning or error in the code has been successfully resolved by the suggested changes, without introducing any other errors.

3.2.2 Evaluation Procedure. Given a set of bugs found by static analyzer tools, we provide the file contents and diagnostic information to the LLM to generate a fix. We assess whether the model fixed the original error, whether it created any new errors, and whether the model-modified code remained syntactically correct after the fix was inserted.

3.3 Code Generation from Natural Language (generate)

This task involves generating a code snippet from a natural language description. Figure 1 shows an example of such a task in the VS Code IDE. In this case, the developer asks the LLM to write a function that produces the first n values of the Fibonacci sequence, and the editor shows the generated function in a diff view.

3.3.1 Metrics. Similar to previous evaluations of code generations [10], we consider a generated code snippet to be successful if the generated code is syntactically correct and all test cases covering the generated code pass. Therefore, we report the following metrics for this scenario:

- *Syntax Correctness*: We compute and report the percentage of generated code that is syntactically correct. For this metric, we check the syntax correctness of the generated code using a language-specific parser (e.g., tree-sitter for each language).
- *Test Pass Rate*: We check the number of passing and failing tests and compute the passing test ratio. To compute this number, we execute the entire test suite

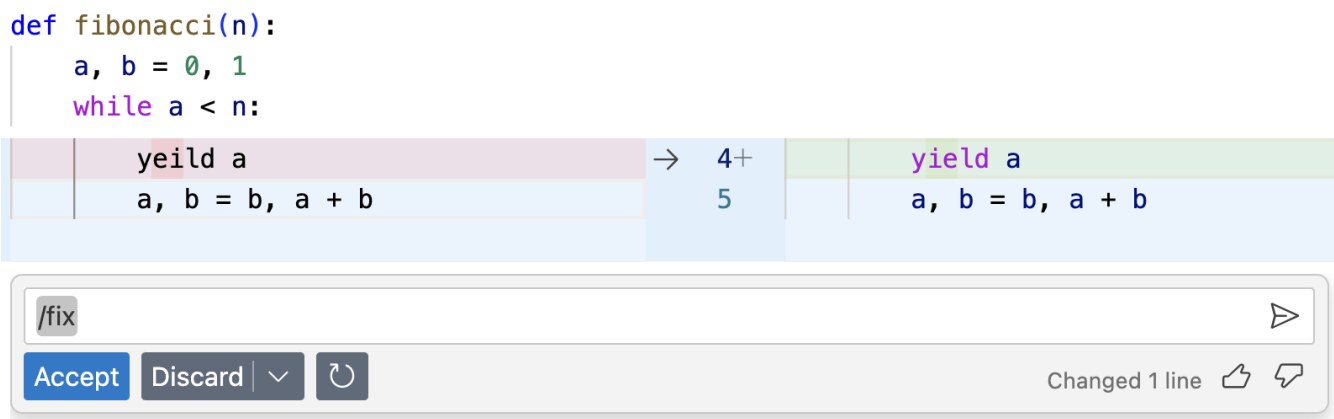

Figure 4. A developer asks the model to fix an error in their fibonacci code, and the model presents the fix (spelling the word "yield" correctly) in diff format.

of the user project and track which tests fail that passed prior to the model's code injection.

3.3.2 Evaluation Procedure. We begin with a set of repositories with test cases. From each repository, we select the methods that are: 1) covered by the test cases in the given repository's test suite, and 2) have a docstring. For each method, we ask an LLM to generate the body of the method given the method's signature and docstring. We provide the contents of method's file as context to the LLM, replacing the original method body with a commented line reading "Your Code Here."

After the LLM generates the method body, we put the generated code back in place of the original method body and evaluate the code by running the repository's test suite against the new method body. We then compute and report the syntax correctness and test pass rate, as explained above.

3.4 Test Case Generation for Code (test)

This task involves using LLMs to generate test cases for code. Developers usually shortcut when it comes to writing unit tests. Automating test generation can motivate more developers to include unit tests. Figure 5 shows an example of a developer requesting tests in the VS Code IDE. In the example case, the developer asks the LLM to generate a test for a Fibonacci function using the /test chat scenario command.

3.4.1 Metrics. In this scenario, we consider a generated test to be successful if it is syntactically correct and can pass on execution. Note that, for this evaluation, this means we assume the code for which the test was written is correct.

- *Syntax Correctness*: We compute the percentage of generated tests that are syntactically correct. We check the syntax correctness of the generated tests using a language-specific parser.
- *Generated Test Pass Rate*: We compute the pass rate of the generated test. We assume the original method is correct, and execute the generated test on its focal method.

3.4.2 Evaluation Procedure. Given a set of methods, we provide the method signature, docstring, and body as context to the LLM to generate a test for each focal method.

Once the LLM generates a test for the method, we add the test to the repository containing the method, and attempt to execute the test.

For Javascript and Typescript, we generate tests using either the Jest or Mocha library. The original test suite of the repository does not need to be written with either library, but each method's original file must be able to pass without errors when a trivial test case (which essentially just asserts true) is appended to the file. When evaluating the generated tests, we temporarily append them to the focal method's file to mitigate import errors, and run the entire file. If running the file with a trivial test case appended (e.g. a test that should always be true) returns false or an error, we know the results from the generated test on that file are not reliable.

3.5 Workspace Understanding and Query Resolution (workspace)

In the Workspace task, we give the model a user's natural language query, and ask it to identify relevant snippets of the codebase that may aid in answering the user's question. This tests a model's ability to comprehend both natural language requests from a user and large amounts of code.

3.5.1 Metrics. We evaluate the quality of an LLM's retrieved snippets in two ways:

- Mean Reciprocal Rank (MRR): Given a ranked list of the model's retrieved snippets, we calculate $\frac{1}{r}$, where r is the rank of the correct snippet in the model's list. So, if the model ranks the correct snippet second, we

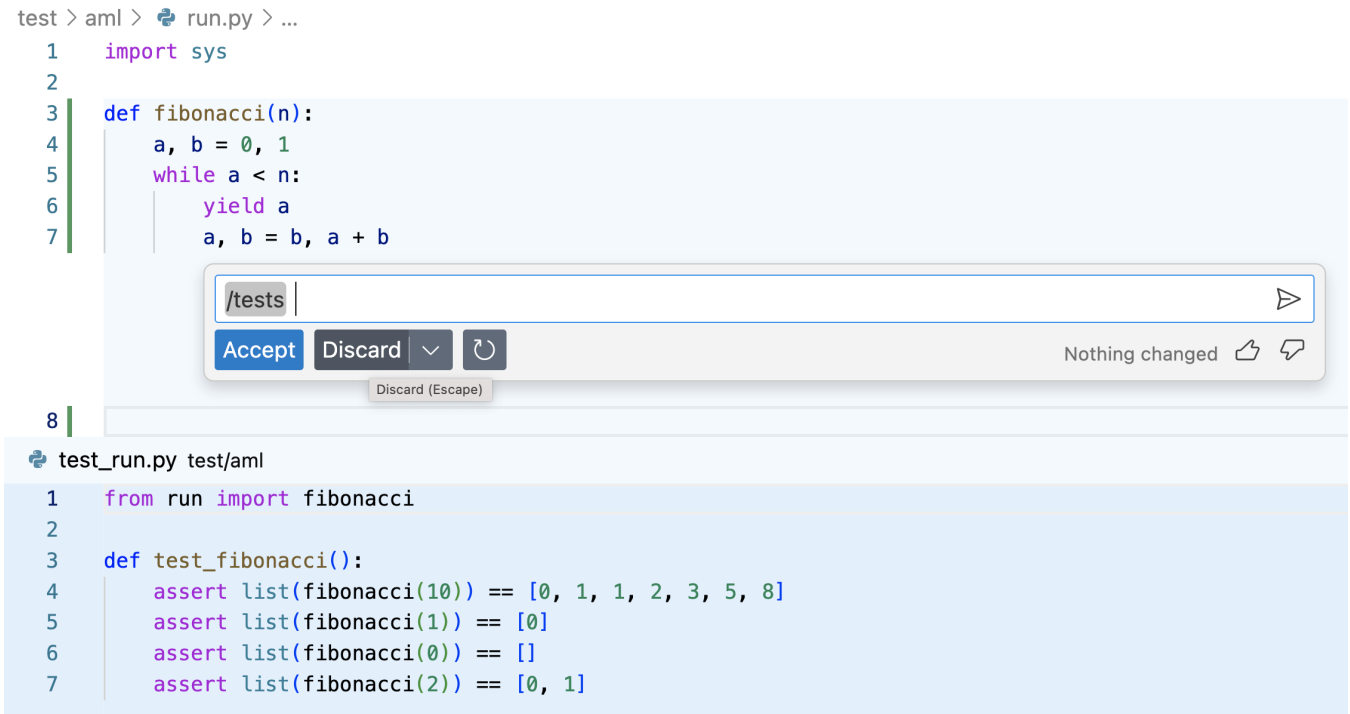

Figure 5. A developer uses /test to generate a test for a function that generates Fibonacci numbers. The LLM generates the test_fibonacci function for this function in a test file.

would consider the model's score for that test case to be $\frac{1}{2}$. MRR is the mean of all the test case scores.

- End to End Keyword Detection: We begin with a manually created dataset of user queries and keywords associated with the correct answer to the query. We take the model's ranked list of retrieved snippets and pass it to the model along with each user query. Then, we detect whether or not the associated keyword appeared in the model's response, given both the query and the retrieved results.

3.5.2 Evaluation Procedure. For each datapoint, we provide the LLM with a user query and the full context of the codebase associated with the given query. We ask the LLM to retrieve a ranked list of relevant code snippets from the codebase. We directly evaluate the quality of the model's retrieved results using MRR, a metric that scores how well models are able to find the most relevant code snippets during retrieval.

We also evaluate the quality of all the retrieved code snippets by asking the model to answer the original user query, providing the query and the snippets as context. We search the model's final response for a set of keywords associated with the given query to determine whether or not the model was able to find the information it needed to fully answer the question.

With this metric, we evaluate the model's retrieval abilities on an end to end scale, and determine a model's skill at finding code snippets that would actually help it answer the question at hand.

4 Copilot Evaluation Harness

We introduce the end to end Copilot Evaluation Harness for computing evaluation metrics as described above. First, we share the details of collecting the data required for each evaluation. Then, we explain the process of creating a test environment given each language and the need to build and run tests. Finally, we give additional specific implementation details about the evaluation process for each metric.

4.1 Data Collection

Our dataset is made up of methods from hundreds of public GitHub repositories across 6 languages: JavaScript, Typescript, Python, Java, C/C++, and C#. Some of our evaluations require the ability to build and run tests for repositories associated with test cases. To meet this requirement, we have developed a build agent as part of our evaluation harness that attempts various build and test strategies on any arbitrary repository. In addition, we have the capability to run static analysis tools on the repositories that we can build and test. This build agent is essential in collecting the test datasets and performing evaluations.

For each language, we sample from Github public repositories whose code we are able to build and whose test suites

we are able to run using our build agent. The build agent supports Node 18+, Python 3.8+, Java JDK 1.8 (requiring Maven), .NET 6.0, 7.0 and 8.0, and a manually curated set of C++ repositories. We resorted to manually gathering C++ repositories due to the wide variability of C++ build steps. We ignore repositories that are smaller than 1 MB and larger than 100 MB. We ignore repositories that take longer than 10 minutes to build and run tests. Lastly, we ignore repositories that do not contain any methods.

4.1.1 Javascript and Typescript. In Javascript and Typescript, we sub-select on repos that contain a *package.json* file at the root directory. The *package.json* file works in concordance with npm (Node Package Manager) to handle various tasks within the repo, such as specifying dependencies for installation and running the test suite. We rely on npm for our evaluation of Javascript and Typescript code, so we only consider repos whose infrastructure is built to be managed with npm.

4.1.2 Java. In Java, we consider repositories that leverage Maven for their build process. In addition, as of writing, we only consider projects that use JDK 1.8.

4.1.3 Python. In Python, we only consider repositories for which we are able to successfully install all dependencies within a virtual environment.

4.1.4 C/C++. In C/C++, we leverage clang for building projects. Because of the sheer variety of ways that C/C++ repositories can be built, we present a set of manually curated repositories which we have verified will build and test within a docker image.

4.2 Test Case Collection

After identifying suitable repositories for each language, we generate test cases for each evaluation metric based on the code within the repositories. Most evaluations require identifying methods that meet certain conditions, such as being covered by existing tests or containing a warning from a static analysis tool. The criteria for generating evaluation test cases varies from metric to metric, and is explained for each metric below.

4.2.1 Documentation Generation from Code (doc). We create test cases by identifying methods in the repository that are longer than three lines and are not a result of minification or obfuscation. We provide the method and ask the coding assistant being evaluated to generate a docstring for the method. We consider a docstring generation to be successful if the location, format, and coverage of the generated text is correct.

4.2.2 Bug Fixing (fix). We create test cases based on static analysis tool warnings and errors flagged on a given repository. We only consider static analysis warnings that are not related to imports or configuration because such issues are difficult to fix with only a single file as context. We consider a generated fix to be successful if it is syntactically correct and strictly reduces the numbers of static analysis warnings on execution. We must consider a strict decrease rather than the presence of the original warning or error, because it is possible for the coding assistant to fix the original issue while introducing a new issue, which a developer would not look upon as a complete fix.

4.2.3 Code Generation from Natural Language (generate). We create test cases by identifying methods in a given repository that are covered by some existing passing test. The test case gives the coding assistant visibility of the entire file up to and including the method signature. The coding assistant is then asked to generate the method body associated with the method signature. We consider a generated code snippet to be successful if the generated code is syntactically correct and all test cases covering the generated code pass.

4.2.4 Test Generation from Code (test). We create test cases by identifying methods within a given repository. We ask the coding assistant to provide a working test for the given method. We consider the generated test to be successful if it invokes the given method and passes execution.

4.2.5 Workspace Understanding and Query Resolution (workspace). We collected questions from developers about certain aspects of their project workspace, such as the idiomatic way to build a certain feature. The context fetching that takes place as a part of the workspace command will return several related code snippets. We evaluate the quality of an LLM's retrieved snippets using MRR as explained above.

5 Experiments

Using Copilot evaluation harness metrics and the test cases detailed above, we compute the success of two OpenAI models: GPT-3.5 and GPT-4, as well as CodeLlama on the document generation and bug fixing scenarios using the an LLM powered chat extension in VSCode IDE with more than 700K active users as the code assistant.

Our experimentation aims to answer the following research questions:

- **RQ1. Model Comparison**: How do different LLM's compare to one another when integrated with a coding assistant?
- **RQ2. Integration Improvements**: What insights can the Copilot Evaluation harness provide engineers to improve the integration of LLM in a coding assistant?
- **RQ3. Data Validity**: How do our evaluation test cases compare with actual usage of a LLM powered coding assistant? Do the test cases in our harness reflect how real-world users interact with a LLM powered coding assistant?

In this section, we discuss findings pertaining to these research questions.

Doc			
Language	**Model**	**Syntax Correctness**	**Format Correctness**
Python	GPT-4	100%	83%
	GPT-3.5	100%	87%
	CodeLlama	100%	87%
Javascript	GPT-4	83%	100%
	GPT-3.5	83%	100%
	CodeLlama	79%	55%
Typescript	GPT-4	96%	79%
	GPT-3.5	96%	86%
	CodeLlama	77%	65%
Java	GPT-4	100%	93%
	GPT-3.5	100%	80%
	CodeLlama	100%	64%
C#	GPT-4	100%	89%
	GPT-3.5	100%	75%
	CodeLlama	94%	67%
C/C++	GPT-4	92%	94%
	GPT-3.5	92%	77%
	CodeLlama	90%	38%

Table 1. LLMs performance on the Doc chat scenario across Python, Javascript, Typescript, Java, C# and C/C++ for the specific success metrics of the Doc scenario.

Fix			
Language	**Model**	**Syntax Correctness**	**Bugs Fixed**
Python	GPT-4	96%	74%
	GPT-3.5	93%	68%
	CodeLlama	88%	39%
Javascript	GPT-4	92%	81%
	GPT-3.5	85%	74%
	CodeLlama	39%	26%
Typescript	GPT-4	83%	75%
	GPT-3.5	74%	75%
	CodeLlama	70%	30%
C#	GPT-4	98%	58%
	GPT-3.5	96%	65%
	CodeLlama	84%	50%

Table 2. LLMs performance on the Fix chat scenario across Python, Javascript, Typescript and C# for the specific Fix success metrics.

5.1 RQ1. Model Comparison

Below we discuss our learnings comparing three state of the art LLMs when used to power our target chat extension in VSCode.

5.1.1 Documentation Generation from Code (doc). Table 1 shows that, for docstring generation, GPT-4 generally outperforms GPT-3.5 and Code Llama. GPT-3.5 and GPT-4 are very similar in performance to one another, with Code Llama slightly behind. The key exceptions here are Python, where Code Llama performs at a slightly higher level than GPT-4, and C/C++, where Code Llama performs significantly worse. One possible explanation is that GPT-3.5 and GPT-4 were trained on a massive corpus, encompassing much of the open source code on the internet. As such, the GPT models' performance could be inflated by the fact that it has seen many different code patterns. Code Llama, a comparatively small model, is much less likely to have seen a given code snippet, potentially hindering its performance in comparison to the GPT models.

Passing Fix Request

```
You have been given the file contents of  lightning_train_net.py.
The following code snippet within the file has a bug:
def training_epoch_end(self, training_step_outputs):
    self.iteration_timer.after_train()
    if comm.is_main_process():
        self.checkpointer.save("model_final")
    for writer in self.writers:
        writer.write()
        writer.close()
    self.storage.__exit__(None, None, None)
This is the line with the error:
for writer in self.writers:
This is the problem with the line:
Object of type "None" cannot be used as iterable.
Describe in a single sentence how you would solve the
problem. Then, fix the error.
```

GPT-4 Passed Response

```
To solve the problem, I would add a condition to check
if `self.writers` is not None before iterating over it.
def training_epoch_end(self, training_step_outputs):
self.iteration_timer.after_train()
if comm.is_main_process():
    self.checkpointer.save("model_final")
if self.writers is not None:
    for writer in self.writers:
        writer.write()
        writer.close()
self.storage.__exit__(None, None, None)
```

Figure 6. Example prompt and response for a passing fix test case.

Failed Fix Request

```
You have been given the file contents of tutorial009.py.
The following code snippet within the file has a bug:
def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(or_(
        Hero.age <= 35, Hero.age > 90
        ))
        results = session.exec(statement)
        for hero in results:
            print(hero)

def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
This is the line with the error:
statement = select(Hero).where(or_(
            Hero.age <= 35, Hero.age > 90
            ))
This is the problem with the line:
Operator ">" not supported for "None".
Describe in a single sentence how you would solve the
problem. Then, fix the error.
```

GPT-4 Failed Response

```
The problem arises because the `age` field of the `Hero`
model is optional and can be `None`, and the
comparison operator `>` is not supported for `None`.
To fix this, we need to add a condition to check if
`Hero.age` is not `None` before comparing it with a
number.
def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(or_(
        Hero.age <= 35, Hero.age > 90, Hero.age.isnot(None)
        ))
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
```

Figure 7. Example prompt and response for the failed fix chat scenario. Here, the model determines what the problem might be and attempts to fix it. However, the fix is not correct, and the same syntax error still exists, because it tries to do the > operation before checking whether the age is None.

5.1.2 Bug-Fixing (fix). Table 2 shows the results for bug fixing: similarly to the docstring generation evaluation, GPT-4 tends to slightly outperform GPT-3.5, with Code Llama further behind. For bug fixing, the exception seems to be C#, for which all three models seem to struggle, with GPT-3.5 ultimately outperforming both GPT-4 and Code Llama. Figures 6 and 7 show a passed and failed example with GPT-4. In Figure 6, the model adds an if statement to check whether $self.writers$ is None before trying to use it as an iterable, which solves the error. In Figure 7, the model similarly adds a check for whether *Hero.age* is None. However, since it adds the check after the operation that is causing the error, the error continues to occur. Although the model was able to identify a potential fix, it inserted the check in the wrong location and was not able to fix the bug.

GPT-3.5 vs. GPT-4 Fix Request Response

```
You have been given the file contents of picgoCoreIPC.ts
The following code snippet within the file has a bug:
  const handler = (msg: string) => {
    if (msg === 'NPM is not installed') {
      dialog.showMessageBox({
        title: T('TIPS_ERROR'),
        message: T('TIPS_INSTALL_NODE_AND_RELOAD_PICGO'),
        buttons: ['Yes']
      }).then((res) => {
        if (res.response === 0) {
          shell.openExternal('https://nodejs.org/')
        }})}}
This is the line with the error:
      }).then((res) => {
This is the problem with the line:
Parameter 'res' implicitly has an 'any' type.
Describe in a single sentence how you would solve the problem.
Then, fix the error.
```

GPT-3.5 Passing Response

```
To fix the issue of the implicitly typed parameter 'res', you can
explicitly type it as 'any' in the arrow function
  const handler = (msg: string) => {
    if (msg === 'NPM is not installed') {
      dialog.showMessageBox({
        title: T('TIPS_ERROR'),
        message: T('TIPS_INSTALL_NODE_AND_RELOAD_PICGO'),
        buttons: ['Yes']
      }).then((res: any) => { // Explicitly type 'res' as 'any'
        if (res.response === 0) {
          shell.openExternal('https://nodejs.org/')
        }})}}
```

GPT-4 Failing Response

```
To solve the problem, I would explicitly define the type of 'res'
in the promise callback
  const handler = (msg: string) => {
    if (msg === 'NPM is not installed') {
      dialog.showMessageBox({
        title: T('TIPS_ERROR'),
        message: T('TIPS_INSTALL_NODE_AND_RELOAD_PICGO'),
        buttons: ['Yes']
      }).then((res: Electron.MessageBoxReturnValue) => {
        if (res.response === 0) {
          shell.openExternal('https://nodejs.org/')
        }})}}
```

Figure 8. Example prompt and response for the same test case, where both GPT-3.5 and GPT-4 should fail, but only GPT-4 fails because it attempts a more nuanced approach to fixing the bug than GPT-3.5.

A common cause of differing results between the GPT-3.5 and GPT-4 LLMs occurs when the models try to resolve the "has an 'any' type" error, as can be seen in Figure 8. When the

GPT-4 model attempts to specify the type of a variable named *res*, it predicts the type as *Electron.MessageBoxReturnValue* and casts the variable to that type. That type, however, is not a valid return type for the code. GPT-3.5, on the other hand, casts the variable to type *any*, thereby circumventing more complex issues, but leaving in a code smell (since it is not good practice to cast variables to type *any*). In this case, GPT-3.5 passes our evaluation, while GPT-4 fails, even though GPT-4's attempted fix is more nuanced and advanced. Upon closer inspection of the cases where GPT-3.5 succeeds and GPT-4 fails, we see this phenomenon frequently: GPT-4 fails with a more complicated approach, while GPT-3.5 technically passes, but with a rudimentary and sub-optimal solution.

5.2 RQ2. Integration Improvements

Below we discuss how our evaluation harness can be used to learn insights on how to better integrate LLMs with IDEs.

5.2.1 Documentation Generation from Code (doc).

Further inspection of our results in table 1 reveals four classes of errors that cause the docstring generation evaluation to fail:

1. **Code Logic Changes:** The model changes the fundamental logic of the code when it rewrites the focal function to the file along with the generated docstring.
2. **Syntax Changes:** The model changes the syntax of the focal code when writing to the file. This includes changes such as adding semicolons to the end of lines, or adding type decorators to the function signature.
3. **Incomplete Docstrings:** The model generates a description for the correct function, but does not describe the returned object and every parameter of the function.
4. **Irrelevant Docstrings:** The model returns a docstring that does not pertain to the code block we asked it to document.

Upon closer inspection, in cases where only one of GPT-3.5 or GPT-4 passes, we notice that the GPT-4 model is more likely to make changes to the focal code that make the code cleaner than GPT-3.5. For example, in Figure 9, the GPT-4 model adds type decorators to the inputs of the function, and specifies the return type. Although the GPT-4 model's docstring is correct (and more detailed than that of the GPT-3.5 model), the GPT-4 model fails this test case, since we expect the model to make no changes to the focal code. However, such an error shows how the GPT-4 model's attempts at a more involved improvement may decrease its score without being indicative of worse performance.

Based on this finding, we inserted an additional instruction in the coding assistant's docstring generation prompt that tells the model specifically not to change any of the focal code. This resulted in a significant improvement in the evaluation results for all the languages, ranging from 5% in C++ to 11% in Java.

Doc: GPT-4 Syntactical Change

```
Write documentation for dump
----------------------------
function dump(classFunction, pref) {
  window.document.write("<b>" + pref + classFunction.name
                          + "</b> <br/>");
  const keys = Object.keys(classFunction);
  if (keys.length > 0 && keys[0] !== "0") {
    for (const prop of keys) {
      dump(classFunction[prop], pref +
                          classFunction.name + ".");
    }}}
```

GPT-3.5 Passing Response

```
/**
 * Recursively dumps the properties of a class or object.
 *
 * @param classFunction - The class or object to dump.
 * @param pref - The prefix to use for indentation.
 */
function dump(classFunction, pref) {
  window.document.write("<b>" + pref + classFunction.name
                          + "</b> <br/>");
  const keys = Object.keys(classFunction);
  if (keys.length > 0 && keys[0] !== "0") {
    for (const prop of keys) {
      dump(classFunction[prop], pref +
                          classFunction.name + ".");
    }}}
```

GPT-4 Failing Response

```
/**
 * Writes the name of the given class function and its
 * properties to the document.
 * If the class function has properties that are also functions,
 * it recursively writes their names and properties as well.
 *
 * @param classFunction - The function to be dumped.
 *              It should be a class or a function.
 * @param pref - The prefix to be added before the function name.
 *         It is used for nested functions to show the hierarchy.
 */
function dump(classFunction: Function, pref: string): void {
  window.document.write("<b>" + pref + classFunction.name
                          + "</b> <br/>");
  const keys = Object.keys(classFunction);
  if (keys.length > 0 && keys[0] !== "0") {
    for (const prop of keys) {
      dump(classFunction[prop], pref +
                          classFunction.name + ".");
    }}}
```

Figure 9. Example prompt and response for doc evaluation. GPT-3.5 passed, and GPT-4 failed because it returned the original function with parameter and return types. This fails our evaluation because we require the model to leave the focal code unchanged.

We also see that GPT-4 is better at following specific instructions. This is highlighted by the example in Figure 10, where GPT-4 generates documentation for the Vec class instead of the *get_colour_at* function. At first, this appears

to be an error on GPT-4's part. However, a review of the prompt shows an incorrect parsing of the function name for C++ erroneously puts Vec in the prompt rather than *get_colour_at*. Even though GPT-3.5 received the same instructions, it wrote documentation for *get_colour_at*. From examples such as this, we see that the GPT-4 model is more sensitive to instructions, and we must design our prompts accordingly.

Doc: Incorrect Focal Function

```
Write documentation for Vec
----------------------------
Vec Material::get_colour_at(double u, double v) const {
    if (m_texture.is_loaded())
        return m_texture.get_pixel(u, v);

    return m_colour;
}
```

GPT-3.5 Passing Response

```
/**
 * @brief Retrieves the color at the specified texture coordinates.
 *
 * @param u The u-coordinate of the texture.
 * @param v The v-coordinate of the texture.
 * @return The color at the specified texture coordinates.
 */
Vec Material::get_colour_at(double u, double v) const {
    if (m_texture.is_loaded())
        return m_texture.get_pixel(u, v);

    return m_colour;
```

GPT-4 Failing Response

```
/**
 * @class Vec
 *
 * Represents a vector in 3D space. This class is typically
 * used to represent points in 3D space or RGB color values.
 *
 * Note: The actual definition of this class
 * is not shown in this file.
 */
Vec Material::get_colour_at(double u, double v) const {
    if (m_texture.is_loaded())
        return m_texture.get_pixel(u, v);

    return m_colour;
```

Figure 10. Example prompt and response for doc evaluation. GPT-3.5 passed, and GPT-4 failed because it added documentation for the Vec class, rather than the get colour function. This is caused by a failure we were able to highlight in the parsing and prompting of VSCode's pipeline: the model is erroneously asked to document Vec, not get colour. GPT-4 failed because it is better at following directions.

With recommendations based on these results from the evaluation harness, we were able to significantly decrease the number of syntax and code logic errors in documentation generation in the LLM powered chat extension for VS Code. We were also able to fix the error in the parsing of functions in Java and C++ that resulted in the model receiving the incorrect keyword as the function name. Discoveries of such errors, as well as the respective improvements, are only possible with a robust and comprehensive evaluation system.

5.2.2 Bug Fixing (fix). To understand the performance of the LLMs on the bug fixing evaluation, we delve deeper into the types of errors our static analyzers surface. Table 3 shows a breakdown of some common static analyzer errors given to the models to solve for Typescript, as well as the GPT models' performance on them. We see that both models are often able to find the namespace of objects and fix type issues. However, as referenced previously, there are still many cases where the "has an 'any' type" error is not correctly resolved. Figure 8 shows such an example. And, while Figure 8 shows an example for GPT-4, Figure 11 shows a similar phenomenon for GPT-3.5. The GPT-3.5 model hallucinates a type *Token* for the *token* variable. Tho tackle this issues in a LLM powered chat extension, we need to provide the models with additional context such as target variable types and namespaces so the LLM may correctly fix this issue, rather than incorrectly using existing types or hallucinating entirely new types.

Fix Errors in Typescript

Bug Type	Model	Bug Fixed
Cannot find name/namespace	GPT-4	81%
	GPT-3.5	70%
Has an 'any' type	GPT-4	79%
	GPT-3.5	64%
Is not generic	GPT-4	50%
	GPT-3.5	54%

Table 3. Breakdown of common types of errors detected by the static analyzer for Typescript, as well as the relative performance of the GPT models on each type of error.

5.3 RQ3. Data Validation

Although our dataset are pulled from real-world git repositories, this does not guarantee that our test cases accurately reflect how users interact with LLMs through IDEs. To confirm the validity of our dataset, we gather usage data that illustrates how hundreds of Microsoft developers use the docstring generation and bug fixing functionalities of our target LLM powered chat extension in VS Code. We then compare these instances to our test cases.

Fix Hallucination - GPT-3.5

```
You have been given the file contents of Parser.ts
The following code snippet within the file has a bug:
private get categorizedTokens() {
    /**
     * @link http://stackoverflow.com/a/10939280/1149074
     */
    const parameterAssignmentTokens = _.takeWhile(
            this.tokens, token => token.value.includes("="));

    const commandWordToken = this.tokens[
                parameterAssignmentTokens.length];

    const beforeArgumentListTokensCount =
                parameterAssignmentTokens.length + 1;
    const argumentListTokens = _.takeWhile(this.tokens.slice
                    (beforeArgumentListTokensCount), token => !(
        token instanceof Scanner.InputRedirectionSymbol
        token instanceof Scanner.OutputRedirectionSymbol
        token instanceof Scanner.AppendingOutputRedirectionSymbol
    ));
    ...
This is the line with the error:
    const parameterAssignmentTokens = _.takeWhile(
            this.tokens, token => token.value.includes("="));
This is the problem with the line:
Parameter 'token' implicitly has an 'any' type.
Describe in a single sentence how you would solve the problem.
Then, fix the error.
```

GPT-3.5 Failing Response

```
To fix the problem of the implicitly typed 'token' parameter,
I would add type annotations to the parameters of the
arrow functions in the code block
private get categorizedTokens() {
    /**
     * @link http://stackoverflow.com/a/10939280/1149074
     */
    const parameterAssignmentTokens = _.takeWhile(
            this.tokens, (token: Token) => token.value.includes("="));

    const commandWordToken = this.tokens[
                parameterAssignmentTokens.length];

    const beforeArgumentListTokensCount =
                parameterAssignmentTokens.length + 1;
    const argumentListTokens = _.takeWhile(this.tokens.slice
                    (beforeArgumentListTokensCount), (token: Token) => !
        token instanceof Scanner.InputRedirectionSymbol
        token instanceof Scanner.OutputRedirectionSymbol
        token instanceof Scanner.AppendingOutputRedirectionSymbol
    ));
    ...
```

Figure 11. Example failing prompt and response from GPT-3.5. The model hallucinates a type (Token) that matches the name of the variable.

For documentation generation, we use OpenAI's ada embedding model [3] to embed the documented code snippets and compare the snippets from our dataset with the one's gathered from Microsoft developers usage data. Similarly, for the bug fixing telemetry, we embed the code snippets that contain the bug. We use PCA dimensionality reduction to plot the data in two dimensions. PCA dimensionality reduction is optimized to find a plane that maximizes the distance between points and outliers. Figures 12 and 13 show the results of this comparison. We see that each language forms a cluster, and the real usage and our data exist within a similar space for each language cluster, for both documentation generation and bug fixing.

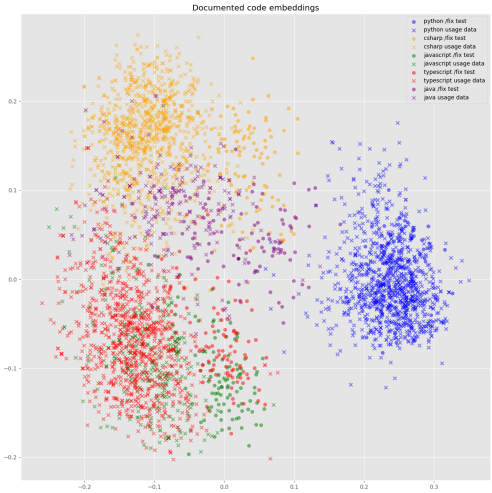

Figure 12. Comparing our dataset for Documentation generation evaluation with real-world usage across languages.

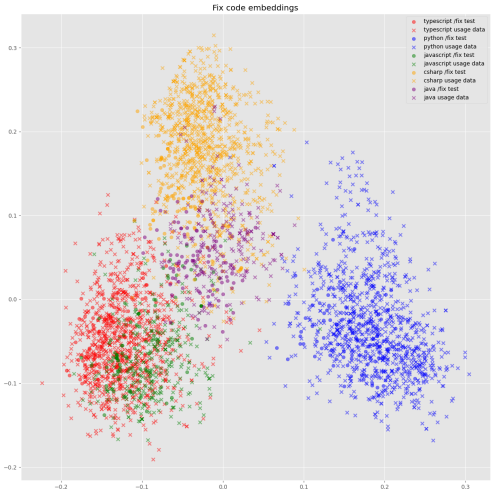

Figure 13. Comparing our dataset for Bug Fixing evaluation with real-world usage across languages.

We do not aim to match the test cases in our dataset to the real usage point for point. Rather, we are determining

whether or not our test cases are outliers in the space of the real usage. If they are not outliers, we can infer that our dataset is in line with the real-world usage of the chat extension. This analysis suggests that our dataset for both the documentation generation and bug fixing evaluation is in line with real world usage.

6 Conclusion and Future Work

With the growing use of LLMs to aid developers in complex engineering tasks comes the need for more robust evaluations of LLM-generated code. Especially as more companies and products seek to integrate LLMs into their workflows, existing evaluation metrics are not sufficient to confirm the quality and correctness of machine-generated code. In this paper, we propose a solution to this problem via the Copilot Evaluation harness. We define five key evaluation metrics for the code generation problem space: method generation, test generation, docstring generation, bug fixing and workspace understanding. We detail the methodology required to collect test cases and evaluation results for each of those five metrics. We also provide preliminary results for two of the five metrics across myriad programming languages.

Our goal in creating the evaluation harness is to validate the quality of LLM-generated code. Although we have seen immense advancements in the code generation ML space, we seek to highlight how much oversight and engineering effort is required to reliably and optimally integrate LLMs into a code workflow. We aim to provide developers a comprehensive evaluation suite, with which they can optimize the integration of LLMs into their coding workflows. With the Copilot Evaluation harness, programmers can more systematically and robustly evaluate the impact of parameters such as prompt wordings, changes in the order of information provided, changes in the context provided to the model, and more.

Moreover, the Copilot Evaluation harness can be used for cost optimizations by revealing that a more budget-friendly LLM model (e.g. CodeLLama) might exhibit satisfactory performance in tasks like documentation. This insight enables developers to intelligently balance resources by allocating tasks to the cost-effective LLM when its performance is deemed sufficient. Simultaneously, more complex tasks can be shifted to more powerful LLMs to ensure optimal outcomes.

We publish this paper as a living documentation of our progress. Future work on this project involves reporting results for the remaining three evaluation metrics and open-sourcing our data and evaluation code.

References

[1] Clang. https://clang.llvm.org/.
[2] Eslint. https://eslint.org.
[3] Openai ada. https://openai.com/blog/new-and-improved-embedding-model.
[4] Pylint. https://pylint.pycqa.org/en/latest/user_guide/usage/run.html.
[5] Pyright. https://github.com/microsoft/pyright.
[6] Roslyn. https://github.com/dotnet/roslyn-analyzers.
[7] Spotbugs. https://spotbugs.github.io/.
[8] Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., Ye, W., Zhang, Y., Chang, Y., Yu, P. S., Yang, Q., and Xie, X. A survey on evaluation of large language models, 2023.
[9] Chen, B., Mustakin, N., Hoang, A., Fuad, S., and Wong, D. Vscuda: Llm based cuda extension for visual studio code. In *Proceedings of the SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis* (New York, NY, USA, 2023), SC-W '23, Association for Computing Machinery, p. 11–17.
[10] Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., et al. Evaluating large language models trained on code.
[11] Chen, Y., Wang, R., Jiang, H., Shi, S., and Xu, R. Exploring the use of large language models for reference-free text quality evaluation: An empirical study, 2023.
[12] Chowdhery, A., Narang, S., Devlin, J., Bosma, M., and Others. Palm: Scaling language modeling with pathways, 2022.
[13] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
[14] Floridi, L., and Chiriatti, M. Gpt-3: Its nature, scope, limits, and consequences. *Minds and Machines 30* (2020), 681–694.
[15] Fu, J., Ng, S.-K., Jiang, Z., and Liu, P. Gptscore: Evaluate as you desire, 2023.
[16] Gao, T., Fisch, A., and Chen, D. Making pre-trained language models better few-shot learners, 2021.
[17] Hendrycks, D., Burns, C., Basart, S., Critch, A., Li, J., Song, D., and Steinhardt, J. Aligning ai with shared human values, 2023.
[18] Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding, 2021.
[19] Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., and Wang, H. Large language models for software engineering: A systematic literature review. *arXiv preprint arXiv:2308.10620* (2023).
[20] Kombrink, S., Mikolov, T., Karafiát, M., and Burget, L. Recurrent neural network based language modeling in meeting recognition. In *Interspeech* (2011), vol. 11, pp. 2877–2880.
[21] Laskar, M. T. R., Bari, M. S., Rahman, M., Bhuiyan, M. A. H., Joty, S., and Huang, J. X. A systematic study and comprehensive evaluation of chatgpt on benchmark datasets, 2023.
[22] Liang, P., Bommasani, R., Lee, T., Tsipras, D., and Others. Holistic evaluation of language models, 2023.
[23] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts, 2023.
[24] Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D., Li, G., Zhou, L., Shou, L., Zhou, L., Tufano, M., Gong, M., Zhou, M., Duan, N., Sundaresan, N., Deng, S. K., Fu, S., and Liu, S. Codexglue: A machine learning benchmark dataset for code understanding and generation, 2021.
[25] Nam, D., Macvean, A., Hellendoorn, V., Vasilescu, B., and Myers, B. In-ide generation-based information support with a large language model, 2023.
[26] OpenAI. Gpt 3.5 models, 2023.
[27] OpenAI. Gpt-4 technical report, 2023.
[28] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems 35* (2022), 27730–27744.

[29] Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics* (2002), pp. 311–318.

[30] Petroni, F., Lewis, P., Piktus, A., Rocktäschel, T., Wu, Y., Miller, A. H., and Riedel, S. How context affects language models' factual predictions, 2020.

[31] Qin, C., Zhang, A., Zhang, Z., Chen, J., Yasunaga, M., and Yang, D. Is chatgpt a general-purpose natural language processing task solver?, 2023.

[32] Rae, J. W., Borgeaud, S., Cai, T., Millican, K., and Others. Scaling language models: Methods, analysis & insights from training gopher, 2022.

[33] Ram, O., Levine, Y., Dalmedigos, I., Muhlgay, D., Shashua, A., Leyton-Brown, K., and Shoham, Y. In-context retrieval-augmented language models, 2023.

[34] Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., Sundaresan, N., Zhou, M., Blanco, A., and Ma, S. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).

[35] Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin, J., et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[36] Roziere, B., Lachaux, M.-A., Chanussot, L., and Lample, G. Unsupervised translation of programming languages. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Red Hook, NY, USA, 2020), NIPS'20, Curran Associates Inc.

[37] Smith, S., Patwary, M., Norick, B., LeGresley, P., Rajbhandari, S., Casper, J., Liu, Z., Prabhumoye, S., Zerveas, G., Korthikanti, V., Zhang, E., Child, R., Aminabadi, R. Y., Bernauer, J., Song, X., Shoeybi, M., He, Y., Houston, M., Tiwary, S., and Catanzaro, B. Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model, 2022.

[38] Sridhar, A., Lo, R., Xu, F. F., Zhu, H., and Zhou, S. Hierarchical prompting assists large language model on web navigation, 2023.

[39] Srivastava, A., Rastogi, A., Rao, A., Shoeb, A. A. M., et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models, 2023.

[40] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems 30* (2017).

[41] Wang, A., Pruksachatkun, Y., Nangia, N., Singh, A., Michael, J., Hill, F., Levy, O., and Bowman, S. R. *SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems*. Curran Associates Inc., Red Hook, NY, USA, 2019.

[42] Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., and Bowman, S. R. Glue: A multi-task benchmark and analysis platform for natural language understanding, 2019.

[43] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models, 2023.

[44] Zhang, X., Yu, B., Yu, H., Lv, Y., Liu, T., Huang, F., Xu, H., and Li, Y. Wider and deeper llm networks are fairer llm evaluators, 2023.

[45] Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., Zhang, S., Ghosh, G., Lewis, M., Zettlemoyer, L., and Levy, O. Lima: Less is more for alignment, 2023.